

3D Shape Parametric Representation

Project Instructions

Due Date: 23:59, 11th December, 2025

Total Points: 100

Submission: Canvas submission of a zip file containing:

- Python implementation
- Generated outputs
- Project report (PDF)
- Presentation slides (PDF)

Late Policy: Late submissions will incur penalties as per course policy.

Overview

In this project, you will focus on the parametric representation of 3D shapes by fitting a 3D model with one or more primitive shapes. Unlike previous tasks, no code template will be provided. You will be given 5 reference 3D models and tasked with approximating them using a set of chosen primitives, see illustrative examples in fig. 1.

The reference 3D models are available in multiple formats including meshes, point clouds, voxels, and occupancy fields. You can choose which format(s) to use as input. At the end of the project phase, you will be asked to test your code on 3 **unseen** models to benchmark the code performance.

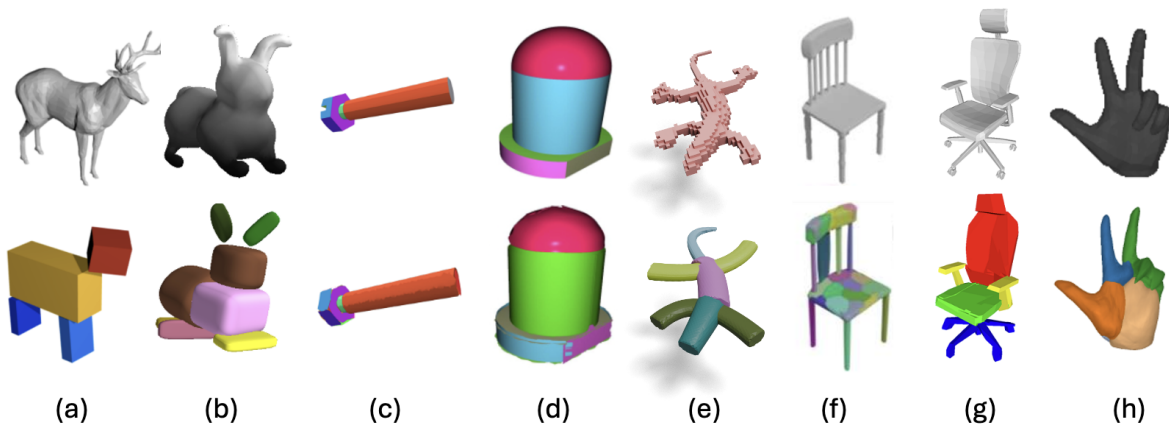


Figure 1: Shape abstractions with different types of primitives. Parametric primitives: (a) Cuboid [8], (b) superquadrics [5], (c) geons [3], and (d) parametric surfaces [7]. (e) sweep surfaces [9]. Neural primitives: (f) Convexes [2], (g) polyhedrons [1], and (h) neural parts by deforming spheres [4]. All figures are adopted from their publication manuscripts.

Objectives

- **Shape Representation:** Choose primitives for representing 3D shapes (e.g., cuboids, superquadrics, cylinders, capsules, spheres, polyhedrons, polylines, prisms, sketch-and-extrude, or custom-designed primitives). You may combine multiple primitive types to achieve better representation quality. You may also conduct CSG operations on primitives.
- **Algorithm Implementation:** Develop a fitting process using either a learning-based method (e.g., neural networks) or a non-learning-based method (e.g., ICP, RANSAC, optimization). Each approach has its trade-offs; choose based on your primitives and objectives. You must demonstrate clear understanding of your chosen method.
- **Evaluation Metrics:** Develop **two** quantitative metrics to evaluate your representation quality. Compare against a baseline method; your results should demonstrate clear improvement over the baseline performance.
- **Ablation Study:** Conduct systematic analysis to justify your design choices. This includes testing alternative components in your pipeline and evaluating their impact both quantitatively and qualitatively.
- **Documentation:** Deliver a technical report (max 10 pages) and presentation of your work. Include comprehensive analysis of your method's strengths and limitations.

Technical Notes

- **External Resources:** You may use existing architectures and code, but you must **cite** all sources and demonstrate thorough understanding of their functionality. Document any modifications made to existing implementations.
- **Primitive Limitations:** No primitive type is perfect for all shapes. You won't be penalized for inherent limitations of your chosen primitives, but you must analyze and explain any representation deficiencies in your report.
- **Baseline Requirements:** You may benchmark against published methods, SphereNet from the assignment, or your own baseline. At minimum, your baseline must implement random primitive initialization on the reference model. Any simpler baseline is unacceptable. Your method should demonstrate clear improvements over the chosen baseline.

Report Guidelines

Required Sections

1. **Abstract** (200 words)

- Problem statement
- Method summary
- Key results

2. **Introduction** (1 page)

- Problem background
- Motivation
- Approach overview
- Main contributions

3. **Related Work** (1-2 pages)

- Primitive-based shape representation
- Shape fitting algorithms
- Relevant evaluation metrics

4. **Method** (2-3 pages)

- Primitive definition and parameterization
- Fitting algorithm details
- Loss functions/optimization objectives
- Implementation details

5. **Experiments** (2-3 pages)

- Dataset description
- Evaluation metrics
- Baseline implementation
- Ablation studies
- Quantitative results
- Qualitative visualizations

6. **Discussion** (1 page)

- Result analysis
- Limitations
- Future improvements

7. **References**

- Complete citations
- IEEE format

Competition and Bonus

Unseen Model Challenge (5 points)

Unseen models will be released one day before the submission deadline.
Performance ranking on unseen test models:

- 1st place: 5 points
- 2nd-4th place: 3 points
- 5th-7th place: 1 point

Marking Rubric (Total 100 Points)

1. Correctness and Implementation (30 points)

Code Implementation (20 points)

Excellent (17-20 pts)	Complete implementation, handles edge cases gracefully, comprehensive documentation, optimized
Good (13-16 pts)	Complete with minor bugs, basic error handling, adequate documentation, generally efficient
Fair (9-12 pts)	Partial implementation with significant bugs, limited error handling, sparse documentation
Poor (0-8 pts)	Incomplete, no error handling, minimal documentation, major performance issues

Code Citation and External Resources (5 points)

Complete (4-5 pts)	All sources cited, detailed explanation of modifications, clear distinction between original/borrowed code
Partial (2-3 pts)	Some citations missing or unclear, limited explanation of modifications
Poor (0-1 pts)	No citations, no explanation of external code usage

Code Organization (5 points)

Excellent (4-5 pts)	Modular structure, consistent naming, well-organized files, clear comments and docstrings
Adequate (2-3 pts)	Basic organization, inconsistent naming, some unclear structure
Poor (0-1 pts)	Disorganized code, no clear structure, poor naming conventions

2. Ablation Study (15 points)

Experiment Design (8 points)

Excellent (7-8 pts)	3+ meaningful variations, clear hypotheses, controlled variables, systematic exploration
Good (5-6 pts)	2 variations tested, basic hypotheses, some variable control
Poor (0-4 pts)	Only 1 or poorly chosen variations, no clear hypotheses, uncontrolled variables

Analysis and Interpretation (7 points)

Excellent (6-7 pts)	Comprehensive analysis, clear graphs/tables, insightful trade-off discussion, well-justified conclusions
Good (4-5 pts)	Basic analysis, simple comparisons, some trade-off discussion
Poor (0-3 pts)	Minimal analysis, missing comparisons, superficial discussion

3. Quantitative Metrics (10 points)

Metric Development (7 points)

Excellent (6-7 pts)	Two well-designed complementary metrics, clear mathematical formulation, justified choice, proper validation
Good (4-5 pts)	Two adequate metrics, basic formulation, some justification
Poor (0-3 pts)	Poorly designed metrics, unclear formulation, no justification

Baseline Comparison (3 points)

Complete (3 pts)	Clear baseline implementation, comprehensive comparison
Partial (2 pts)	Basic baseline comparison
Poor (0-1 pts)	Missing or inadequate baseline

4. Qualitative Results and Report (25 points)

Report Structure and Content (15 points)

Excellent (13-15 pts)	All sections well-developed, clear flow, comprehensive literature review (10+ citations), strong technical depth
Good (10-12 pts)	All sections present, adequate flow, basic literature review (5-9 citations), good technical content
Fair (7-9 pts)	Missing/underdeveloped sections, unclear flow, limited literature review (1-4 citations)
Poor (0-6 pts)	Multiple missing sections, no organization, no literature review, superficial content

Report Formatting (5 points)

Excellent (4-5 pts)	Consistent formatting, professional appearance, all figures/tables numbered and referenced, complete bibliography
Adequate (2-3 pts)	Minor formatting inconsistencies, some reference/citation issues
Poor (0-1 pts)	Major formatting issues, missing or improper citations

Visualization Quality (5 points)

Excellent (4-5 pts)	High-quality clear visualizations, multiple viewpoints, appropriate color/lighting, clear ground truth comparisons
Adequate (2-3 pts)	Basic visualizations, limited viewpoints, some unclear comparisons
Poor (0-1 pts)	Poor quality visualizations, single viewpoint only, missing comparisons

5. Presentation (20 points)**Visual Aids and Delivery (15 points)**

Excellent (13-15 pts)	Professional slides, clear visual hierarchy, effective animations/transitions, engaging delivery, well-paced
Good (10-12 pts)	Good slide design, basic visual hierarchy, some animations, clear delivery, adequate pacing
Fair (6-10 pts)	Basic slide design, unclear hierarchy, limited visual aids, some delivery issues
Poor (0-6 pts)	Poor slide design, no visual hierarchy, no animations/transitions, poor delivery

Q&A Performance (5 points)

Excellent (4-5 pts)	Comprehensive answers, clear technical understanding, handles unexpected questions, professional demeanor
Good (2-3 pts)	Good answers to most questions, solid technical understanding, professional attitude
Poor (0-1 pts)	Unable to answer basic questions, poor technical understanding, unprofessional behavior

FAQs

Performance Expectations and Baselines

Q: Do I need to achieve better performance than existing published methods?

A: No. You are **not** expected to surpass state-of-the-art performance. This is a course project, not a research competition. Your goal is to implement a working system that demonstrates understanding of 3D shape representation concepts. Focus on:

- Creating a functional implementation
- Demonstrating clear improvement over your chosen baseline
- Showing thorough understanding of your approach
- Conducting meaningful ablation studies

Q: What constitutes an acceptable baseline?

A: Your baseline must be more sophisticated than randomly throwing out primitives. Acceptable baselines include:

- Published methods (with proper citation)
- SphereNet from Assignment Task 3
- Your own implementation using simpler approaches (e.g., basic fitting algorithms)
- Simplified versions of your main method

Unacceptable baselines: random primitive placement, trivial approaches without algorithmic merit.

Code Usage and Attribution

Q: Can I use existing code and architectures?

A: Yes, but with strict attribution requirements:

- **Encouraged:** Using existing architectures as building blocks (e.g., PointNet encoder, standard neural network layers, optimization libraries)
- **Required:** Full citation of all external code sources
- **Required:** Clear documentation of what code is borrowed vs. original
- **Required:** Detailed explanation of any modifications made

Example of proper usage: "We adopt the PointNet encoder architecture from [6] for point cloud feature extraction, but implement our own primitive fitting head and loss functions."

Q: What's the difference between acceptable code reuse and plagiarism?

A:

Acceptable:

- Using standard libraries and established architectures as building blocks
- Adopting existing optimization routines and loss functions
- Building upon existing codebases with your own algorithmic approach

Unacceptable (significant point deductions):

- Downloading complete solutions with minimal changes
- Using substantial code without proper attribution
- Reproducing existing methods without original contributions

Q: What constitutes sufficient originality?

A: Your **core approach** should demonstrate thoughtful design choices and implementation effort:

- Thoughtful selection of primitives and parameterizations for your target shapes
- Your own fitting algorithm or meaningful adaptation of existing methods
- Justified choice of loss functions and evaluation metrics
- Original combination of existing techniques
- Custom implementation of your overall pipeline

You don't need to invent new loss functions or primitive types - focus on creating a working system with clear design rationale.

Technical Implementation

Q: What input formats should I support?

A: Support at least one provided format (meshes, point clouds, voxels, occupancy fields). Choose what best suits your approach.

Q: How many primitives should I use?

A: No fixed requirement. Conduct experiments to find the optimal number for your method and target shapes.

Evaluation and Metrics

Q: What makes good evaluation metrics?

A: Your two metrics should capture different aspects (e.g., geometric accuracy and efficiency). Examples: Chamfer distance, volume overlap, surface coverage. Avoid using similar metrics (e.g., L1 and L2 distance).

Q: How should I handle failure cases?

A: Acknowledge and analyze them in your report. This demonstrates understanding of limitations and is valued positively.

Report and Submission

Q: What if my approach doesn't work well on all models?

A: This is expected. Focus on demonstrating that your method works for its intended shape types and provide honest analysis of limitations.

References

- [1] Zhiqin Chen, Andrea Tagliasacchi, and Hao Zhang. Bsp-net: Generating compact meshes via binary space partitioning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 45–54, 2020.
- [2] Boyang Deng, Kyle Genova, Soroosh Yazdani, Sofien Bouaziz, Geoffrey Hinton, and Andrea Tagliasacchi. Cvxnet: Learnable convex decomposition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 31–44, 2020.
- [3] Lingxiao Li, Minhyuk Sung, Anastasia Dubrovina, Li Yi, and Leonidas J Guibas. Supervised fitting of geometric primitives to 3d point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2652–2660, 2019.
- [4] Despoina Paschalidou, Angelos Katharopoulos, Andreas Geiger, and Sanja Fidler. Neural parts: Learning expressive 3d shape abstractions with invertible neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3204–3215, 2021.
- [5] Despoina Paschalidou, Ali Osman Ulusoy, and Andreas Geiger. Superquadrics revisited: Learning 3d shape parsing beyond cuboids. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10344–10353, 2019.
- [6] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 652–660, 2017.
- [7] Gopal Sharma, Difan Liu, Subhransu Maji, Evangelos Kalogerakis, Siddhartha Chaudhuri, and Radomír Měch. Parsenet: A parametric surface fitting network for 3d point clouds. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part VII 16*, pages 261–276. Springer, 2020.
- [8] Shubham Tulsiani, Hao Su, Leonidas J Guibas, Alexei A Efros, and Jitendra Malik. Learning shape abstractions by assembling volumetric primitives. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2635–2643, 2017.
- [9] Mingrui Zhao, Yizhi Wang, Fenggen Yu, Changqing Zou, and Ali Mahdavi-Amiri. Sweepnet: Unsupervised learning shape abstraction via neural sweepers. *arXiv preprint arXiv:2407.06305*, 2024.